

Backend Scheduling Workflow

Scope

This walkthrough focuses only on the backend scheduling code and database model in `BitSchedulerCore`, `BitScheduleServices`, and the API schedule endpoints.

The main backend entry points for schedule operations are in:

- `AspireBitSchedule.ApiService/Features/Schedule/ScheduleEndpoints.cs`
- `BitScheduleServices/Features/Schedule/ScheduleFeatureService.cs`
- `BitSchedulerCore/BitSchedule.cs`
- `BitSchedulerCore/Services/BitScheduleDataService.cs`

Short Version

The current backend schedule model is centered on **resource-specific bitmaps**:

- `BitClient` is the tenant/container.
- `BitResource` is the schedulable thing for that client.
- `BitDay` is the in-memory representation of one day of availability/reservations.
- `BitResourceScheduleRanges` is the main persisted schedule storage for a resource.
- `BitEvent` can optionally reserve or release schedule bits for a resource.

`BitReservations` and direct `BitDays` persistence still exist, but they are mostly part of an older model or compatibility path. The newer code path stores schedule state in `BitResourceScheduleRanges`.

Table Purposes

`BitClient`

File references:

- `BitSchedulerCore/BitClient.cs`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs`

Purpose:

- Represents the owning client/tenant.
- Groups resources, events, reservations, and schedule ranges.

Important relationships:

- One client has many `BitResources`.
- One client has many `BitReservations`.
- One client has many `BitEvents`.
- One client has many `BitResourceScheduleRanges`.

In practice, most backend services currently operate with a hard-coded default client ID of `1` through `BitScheduleFactory`.

BitResource

File references:

- `BitSchedulerCore/BitResource.cs`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs`

Purpose:

- Represents the schedulable entity.
- Can be a person, equipment, meeting room, etc.
- Belongs to one `BitClient`.
- Has one `BitResourceType`.

Why it matters to scheduling:

- The current schedule API requires a valid `BitResourceId`.
- Schedule reads and writes are done per resource, not just per client.

BitResourceType

File references:

- `BitSchedulerCore/BitResourceType.cs`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs`

Purpose:

- Classifies resources.
- Mostly administrative metadata.
- Not part of the schedule math itself.

BitDay

File references:

- `BitSchedulerCore/BitDay.cs`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs`

Purpose:

- Represents one day split into **96 15-minute slots**.
- Stores the day as a bitmap in memory.

How it works:

- `DayData` holds the 96 schedule bits.
- `Metadata` holds extra flags in the upper 32 bits.
- `IsFree` is a metadata shortcut so the code can avoid checking all bits when the day is empty.
- `ReserveRange(startSlot, length)` marks blocks as taken.
- `FreeRange(startSlot, length)` clears them.

Important detail:

- **BitDay** is both a domain object and a legacy table-backed entity.
- In the current resource-based workflow, **BitDay** is the working object the code reads into memory, modifies, and then packs back into **BitResourceScheduleRanges**.

BitResourceScheduleRanges

File references:

- **BitSchedulerCore/BitResourceScheduleRange.cs**
- **BitSchedulerCore/Data/BitScheduleDbContext.cs**
- **BitSchedulerCore/Services/BitScheduleDataService.cs**
- **BitSchedulerCore/Services/BitResourceScheduleRangePayloadConverter.cs**

Purpose:

- This is the main persisted schedule store for a resource.
- Each row belongs to one client and one resource.
- Each row covers a date range and stores the schedule for every day in that range inside one **Payload** byte array.

Columns and meaning:

- **BitClientId**: tenant owner.
- **BitResourceId**: which resource this schedule belongs to.
- **StartDate** / **EndDate**: the date window covered by the payload.
- **Payload**: serialized day-by-day bitmap data.

Why this table exists:

- Instead of storing one row per appointment or one row per day only, the backend stores compressed schedule state for a whole range.
- This makes schedule reads/writes resource-centric and bitmap-centric.

BitReservations

File references:

- **BitSchedulerCore/BitReservation.cs**
- **BitSchedulerCore/Data/BitScheduleDbContext.cs**
- **BitSchedulerCore/Migrations/20260601030052_InitialPostgres.cs**

Purpose:

- Represents one discrete reservation with **Date**, **StartBlock**, and **SlotLength**.

Important observation:

- The current schedule read/write flow does **not** use **BitReservations**.
- **BitScheduleDataService** reads/writes either **BitResourceScheduleRanges** or legacy **BitDays**, not **BitReservations**.
- **BitEventService** also reserves time by modifying schedule bitmaps, not by inserting **BitReservations**.

So the practical role today appears to be:

- legacy data model,
- unused/leftover entity for an older design,
- or a future audit/detail concept that is not currently part of the active scheduling workflow.

BitEvent

File references:

- `BitSchedulerCore/BitEvent.cs`
- `BitSchedulerCore/Services/BitEventService.cs`

Purpose:

- Stores business events assigned to a resource.
- Can optionally reserve schedule bits through `ScheduleBitsReserved`.

Why it matters:

- Events are the main business record.
- Schedule bits are the availability/occupancy mechanism.
- If `ScheduleBitsReserved` is true, creating/updating/deleting an event also updates the resource schedule.

How the Main Pieces Work Together

1. Tenant and resource ownership

`BitClient` owns the overall data space.

Inside a client:

- there are many `BitResources`,
- each resource has its own schedule,
- that schedule is stored in `BitResourceScheduleRanges`.

So the real ownership chain is:

`BitClient -> BitResource -> BitResourceScheduleRanges`

2. `BitDay` is the working schedule unit

Even though persistence is range-based, the schedule logic itself works one day at a time.

The code loads a date span into a `Dictionary<DateTime, BitDay>`, where each `BitDay`:

- knows its date,
- knows its client,
- contains 96 bits for the day,
- can reserve/free slot ranges.

That dictionary lives inside `BitSchedule` as `_scheduleData`.

3. `BitResourceScheduleRanges` is the persisted container

`BitScheduleDataService.LoadScheduleData` does this:

1. Creates empty `BitDay` objects for every requested date.
2. Loads overlapping `BitResourceScheduleRanges` rows for the requested client/resource.
3. Uses `BitResourceScheduleRangePayloadConverter.Deserialize(...)` to turn the payload bytes back into `BitDay` objects.
4. Overlays those days into the in-memory dictionary.

When saving, `BitScheduleDataService.SaveScheduleDataAsync` does the reverse:

1. Groups modified days by a canonical range start.
2. Loads or creates the matching `BitResourceScheduleRange`.
3. Applies the updated `BitDay` values into that range.
4. Serializes the full range back into the `Payload`.
5. Inserts or updates the `BitResourceScheduleRanges` row.

4. `BitReservations` is not part of the active flow

This is the part that is easy to forget when revisiting the code:

- `BitReservation` looks like it should be the core scheduling table.
- It is not what the current backend actually uses to enforce occupancy.

The active schedule path is bitmap-based, not reservation-row-based.

Direct Answer: What is the purpose of `BitResourceScheduleRanges`, and how does it fit with `BitReservations`, `BitDay`, and `BitClient`?

`BitResourceScheduleRanges`

Purpose:

- Persist a resource's schedule over a date range as one serialized payload.

Think of it as:

- the database storage format for the resource's calendar bits.

`BitDay`

Purpose:

- The in-memory day object used to manipulate that schedule.

Think of it as:

- the runtime shape of one day when the backend is reading, reserving, or freeing blocks.

Relationship to `BitResourceScheduleRanges`:

- `BitResourceScheduleRanges` stores many days at once.

- Each day inside that payload becomes a **BitDay** when loaded into memory.

BitClient

Purpose:

- Tenant boundary and ownership root.

Relationship to **BitResourceScheduleRanges**:

- Every range row is scoped to a client and a resource.
- This prevents one client's schedule data from mixing with another's.

BitReservations

Purpose in the current codebase:

- Mostly legacy or inactive relative to the active schedule engine.

Relationship to the others:

- Conceptually, a **BitReservation** describes one booked slice of time.
- But the current backend does not reconstruct or maintain the schedule through this table.
- Instead, the actual occupied/free state lives in the bitmap payloads and the **BitDay** objects derived from them.

Read Flow

Backend path:

- **ScheduleEndpoints**
- **ScheduleFeatureService.ReadSchedule(...)**
- **BitScheduleFactory.Create(...)**
- **BitSchedule.LoadScheduleData()**
- **BitScheduleDataService.LoadScheduleData(...)**

What happens:

1. API receives a schedule request with **BitResourceId** and a date range.
2. **BitSchedule** asks **BitScheduleDataService** for the days.
3. Data service loads overlapping **BitResourceScheduleRanges**.
4. Payloads are deserialized into **BitDay** objects.
5. **BitSchedule.ReadSchedule(...)** filters those days and groups them into **BitMonth** objects for the response.

Write Flow

Backend path for a single day:

- **ScheduleFeatureService.WriteScheduleDayAsync(...)**
- **BitSchedule.WriteDayAsync(...)**
- **BitScheduleDataService.SaveScheduleDataAsync(...)**

What happens:

1. Request identifies resource, date, start time, and end time.
2. `BitSchedule` loads the relevant days into memory.
3. Requested time is converted to block indexes.
4. `BitDay.ReserveRange(...)` marks the bits.
5. Data service finds the proper persisted range row.
6. The full range payload is re-serialized and saved.

Backend path for a repeating schedule write:

- `ScheduleFeatureService.WriteScheduleAsync(...)`
- `BitSchedule.WriteScheduleAsync(...)`
- `BitScheduleDataService.SaveScheduleDataAsync(...)`

What happens:

1. Iterate through loaded `BitDay` objects.
2. Keep only dates in the requested range and active weekdays.
3. Try to reserve each matching day.
4. If any day conflicts, the write is treated as failed and in-memory changes are reverted.
5. If all succeed, persist the modified days back into `BitResourceScheduleRanges`.

Event Flow

Backend path:

- `BitEventService.CreateEventAsync(...)`
- `BitEventService.UpdateEventAsync(...)`
- `BitEventService.DeleteEventAsync(...)`

When `ScheduleBitsReserved` is true:

- create event -> reserve schedule bits,
- update event -> release old bits, then reserve new bits,
- delete event -> release bits.

Important detail:

- Event reservation logic uses `BitScheduleDataService.LoadScheduleData(...)` and `SaveScheduleDataAsync(...)`.
- It does not create `BitReservation` rows.
- So events and schedule occupancy are synchronized through `BitResourceScheduleRanges`.

Serialization Format in `BitResourceScheduleRanges`

`BitResourceScheduleRangePayloadConverter` packs each day into:

- 12 bytes for `DayData` (96 schedule bits),
- 4 bytes for `Metadata`,
- total 16 bytes per day.

So one row's `Payload` is basically:

- day 1 bitmap + metadata,
- day 2 bitmap + metadata,
- day 3 bitmap + metadata,
- and so on for the entire covered date range.

That is why `BitResourceScheduleRanges` can store many days efficiently in one row.

Legacy vs Current Model

Current active schedule model

- Resource-specific.
- Uses `BitResourceId`.
- Persists to `BitResourceScheduleRanges`.
- Uses `BitDay` as the in-memory schedule unit.
- Used by the schedule API and event reservation logic.

Legacy/compatibility model

- Uses `BitDays` directly when `BitResourceId <= 0`.
- `BitScheduleDataService` still contains `LoadLegacyBitDays(...)` and `SaveLegacyBitDaysAsync(...)`.
- `SeedingService` still seeds direct `BitDays`.

Largely inactive model

- `BitReservations` entity/table exists.
- Current backend schedule and event flows do not use it for persistence or conflict tracking.

Important Implementation Notes

1. Canonical range saving uses 6-month buckets

`BitScheduleDataService` uses a constant `RangeMonths = 6`.

That means when saving resource schedules, days are grouped into canonical ranges:

- January 1 through June 30
- July 1 through December 31

This is an important design choice because one row can cover a large block of time for one resource.

2. Seed data does not fully match canonical save ranges

`SeedingService.SeedResourceScheduleRangesAsync(...)` seeds rows using the month start and month end passed in.

But normal save logic uses 6-month canonical ranges.

So if you are inspecting real data, you may see:

- monthly seeded `BitResourceScheduleRanges`,
- and separately created 6-month schedule range rows from normal writes.

That overlap is worth remembering when reading database contents.

3. The schedule API expects resource-based scheduling

`ScheduleFeatureService` rejects schedule requests when `BitResourceId <= 0`.

So the normal backend contract now assumes:

- a schedule belongs to a specific resource,
- not just a client-wide calendar.

Mental Model to Keep in Mind

If you want the simplest way to remember the system:

- `BitClient` = who owns the data
- `BitResource` = whose calendar it is
- `BitDay` = one day of 96 schedule bits in memory
- `BitResourceScheduleRanges` = persisted packed calendar data for that resource
- `BitEvent` = business record that may reserve/release those bits
- `BitReservations` = older row-based reservation idea that is not the active engine

Best Files to Revisit Later

If you only re-open a few files next time, make it these:

- `BitSchedulerCore/Services/BitScheduleDataService.cs`
- `BitSchedulerCore/BitSchedule.cs`
- `BitSchedulerCore/BitDay.cs`
- `BitSchedulerCore/Services/BitResourceScheduleRangePayloadConverter.cs`
- `BitSchedulerCore/Services/BitEventService.cs`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs`